

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 SORTING



ALIF AR RAZAK

109082530009

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Go (sering disebut sebagai **Golang**) adalah [bahasa pemrograman](#) yang dibuat di [Google](#)^[13] pada tahun 2009 oleh [Robert Griesemer](#), [Rob Pike](#), dan [Ken Thompson](#).^[11] Go adalah bahasa pemrograman [sumber terbuka](#) yang mudah, sederhana, efisien. Selain itu, Go memiliki level yang sama dengan [Java](#), Yaitu bahasa pemrograman [yang dikompilasi](#) dan menggunakan [sintaks](#) mirip bahasa [C](#), dengan fitur [pengumpulan sampah](#), penulisan terstruktur, keamanan memori, dan pemrograman yang konkuren serta berurutan.^[14] [Kompiler](#) dan Integrated Development Environment (IDE) lainnya disediakan oleh [Google](#) dari awal secara [bebas](#) dan [sumber terbuka](#).

B. Guided

1.

```
package main

import "fmt"

const nMax int = 4321

type arrInt [nMax]int

func insertionSort(T *arrInt, n int) {
    /* I.S. terdefinisi array T yang berisi n bilangan bulat
       F.S. array T terurut secara mengecil atau descending dengan
       INSERTION SORT */

    var temp, i, j int

    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]

        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
        i = i + 1
    }
}
```

```

}

func main() {
    var data arrInt
    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    insertionSort(&data, n)

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :

```

PS C:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-09_Alif-Ar-Razak\Week13-modul14\Guided\tempCodeRunnerFile.
5
12 5 23 8 1
23 12 8 5 1
PS C:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-

```

2. [judul guided 2]

```

package main

import "fmt"

const nMax int = 100

type arrString [nMax]string

func insertionSortString(T *arrString, n int) {

```

```

var temp string
var i, j int

i = 1
for i <= n-1 {
    j = i
    temp = T[j]

    for j > 0 && temp < T[j-1] {
        T[j] = T[j-1]
        j = j - 1
    }

    T[j] = temp
    i = i + 1
}
}

func main() {
    var data arrString
    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    insertionSortString(&data, n)

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :

```

PS C:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-Razak\Week13-modul14\Guided> go run C:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-Razak\Week13-modul14\Guided\tempCodeRunnerFile.go"
sapi kuda ayam gajah
ayam gajah kuda sapi
PS C:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-Razak\Week13-modul14\Guided>

```

3. [judul guided 3]

```

package main

```

```

import "fmt"

func selecetionSortArray(angka *[5]int){
    var idx_min, i, j int
    for i = 0; i < len(angka) - 1; i++ { //loop luar (iterasi
    sortingnya)
        idx_min = i
        for j = i + 1; j < len(angka); j++ { //loop dalam (mencari
        nilai ekstrim)
            if angka[j] <= angka[idx_min] { //sorting terkecil ke
            terbesar
                idx_min = j
            }
        }
        if idx_min != i {
            angka[i], angka[idx_min] = angka[idx_min], angka[i]
        }
    }
}

func main(){
    var arrAngka [5]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka ke-%d : ", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()

    fmt.Println("=== SEBELUM DISORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }

    fmt.Println("\n=== SETELAH DISORITNG ===")
    selecetionSortArray(&arrAngka)
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }
}

```

Output :

```

PS C:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-Razak\Week13-modul14\Guided> go run "c:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-Razak\Week13-modul14\Guided\SelSortArray.go"
Masukkan data angka ke-0 : 5
Masukkan data angka ke-1 : 3
Masukkan data angka ke-2 : 8
Masukkan data angka ke-3 : 1
Masukkan data angka ke-4 : 9

=== SEBELUM DISORTING ===
5 - 3 - 8 - 1 - 9 -
=== SETELAH DISORTING ===
1 - 3 - 5 - 8 - 9 -

```

4. [judul guided 4]

```

package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK        float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(sMHS)-1; i++ { //loop luar
        idx_min = i
        for j = i + 1; j < len(sMHS); j++ { //loop dalam mencari
            //nilai ekstrim
            if sMHS[j].IPK < sMHS[idx_min].IPK { //sorting terkecil
                //ke terbesar
                idx_min = j
            }
        }
        if idx_min != i {
            sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
        }
    }
}

func main() {
    var structMHS [5]mahasiswa

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan Data Mahasiswa Ke-%d ---\n", i)
        fmt.Print("Nama : ")
        fmt.Scan(&structMHS[i].nama)
        fmt.Print("NIM : ")
    }
}

```

```

        fmt.Scan(&structMHS[i].nim)
        fmt.Print("IPK : ")
        fmt.Scan(&structMHS[i].IPK)
    }
    fmt.Println()

    fmt.Println(" === SEBELUM SORITNG === ")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()

    SelectionSortArray(&structMHS)
    fmt.Println(" === SETELAH SORITNG === ")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()
}

```

Output :

```

NIM : 1098
IPK : 3.9
--- Masukkan Data Mahasiswa Ke-3 ---
Nama : Tian
NIM : 1908
IPK : 3.0
--- Masukkan Data Mahasiswa Ke-4 ---
Nama : Idos
NIM : 1832
IPK : 3.99

```

```

=== SEBELUM SORITNG ===
--- Data Mahasiswa Ke-0 ---
Nama : Alif
NIM : 1090
IPK : 4.00
--- Data Mahasiswa Ke-1 ---
Nama : Budi
NIM : 1080
IPK : 2.90
--- Data Mahasiswa Ke-2 ---
Nama : Danis
NIM : 1098
IPK : 3.90
--- Data Mahasiswa Ke-3 ---
Nama : Tian
NIM : 1908
IPK : 3.00
--- Data Mahasiswa Ke-4 ---
Nama : Idos
NIM : 1832
IPK : 3.99
=====

```

```

=== SETELAH SORITNG ===
--- Data Mahasiswa Ke-0 ---
Nama : Budi
NIM : 1080
IPK : 2.90
--- Data Mahasiswa Ke-1 ---
Nama : Tian
NIM : 1908
IPK : 3.00
--- Data Mahasiswa Ke-2 ---
Nama : Danis
NIM : 1098
IPK : 3.90
--- Data Mahasiswa Ke-3 ---
Nama : Idos
NIM : 1832
IPK : 3.99
--- Data Mahasiswa Ke-4 ---
Nama : Alif
NIM : 1090
IPK : 4.00
=====

```

C. Unguided

1. Soal Unguided (insertion) 1

Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda insertion sort), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya. Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array. Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah

status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

```
package main

import "fmt"

type arrInt [1000000]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int
    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]
        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }
        T[j] = temp
        i = i + 1
    }
}

func main() {
    var arr arrInt
    var n int
    n = 0

    var x int
    fmt.Scan(&x)
    for x >= 0 {
        arr[n] = x
        n = n + 1
        fmt.Scan(&x)
    }

    insertionSort(&arr, n)

    var i int
    i = 0
    for i < n {
        if i > 0 {
            fmt.Print(" ")
        }
    }
}
```

```

    }
    fmt.Print(arr[i])
    i = i + 1
}
fmt.Println()

if n <= 1 {
    fmt.Println("Data berjarak tidak tetap")
    return
}

jarak := arr[1] - arr[0]
sama := true
i = 2
for i < n {
    if arr[i]-arr[i-1] != jarak {
        sama = false
    }
    i = i + 1
}

if sama {
    fmt.Println("Data berjarak", jarak)
} else {
    fmt.Println("Data berjarak tidak tetap")
}
}

```

Output :

```

PS C:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-Razak\Week13-modul14\Unguided> go run "c:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-Razak\Week13-modul14\Unguided\insertion1.go"
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS C:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-Razak\Week13-modul14\Unguided> go run "c:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-Razak\Week13-modul14\Unguided\insertion1.go"
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap

```

Penjelasan :

Program diatas bertujuan untuk membaca integer sampai ketemu angka negatif. Array di-sort ascending pakai insertion sort. Setelah terurut, cek apakah selisih antar elemen berurutan selalu sama. Cetak "Data berjarak x" jika iya, "Data berjarak tidak tetap" jika tidak.

2. Soal Unguided (insertion) 2

Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```
package main

import "fmt"

const nMax = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

type DaftarBuku [nMax]Buku

var pustaka DaftarBuku
var nPustaka int

func DaftarkanBuku() {
    fmt.Scan(&nPustaka)
    var i int
    i = 0
    for i < nPustaka {
        fmt.Scan(&pustaka[i].id)
        fmt.Scan(&pustaka[i].judul)
        fmt.Scan(&pustaka[i].penulis)
        fmt.Scan(&pustaka[i].penerbit)
        fmt.Scan(&pustaka[i].eksemplar)
        fmt.Scan(&pustaka[i].tahun)
        fmt.Scan(&pustaka[i].rating)
        i = i + 1
    }
}

func CetakTerfavorit() {
    idx := 0
    var i int
    i = 1
    for i < nPustaka {
        if pustaka[i].rating > pustaka[idx].rating {
```

```

        idx = i
    }
    i = i + 1
}
fmt.Println(pustaka[idx].judul, pustaka[idx].penulis,
pustaka[idx].penerbit, pustaka[idx].tahun)
}

func UrutBuku() {
    var temp Buku
    var i, j int
    i = 1
    for i <= nPustaka-1 {
        j = i
        temp = pustaka[j]
        for j > 0 && temp.rating > pustaka[j-1].rating {
            pustaka[j] = pustaka[j-1]
            j = j - 1
        }
        pustaka[j] = temp
        i = i + 1
    }
}

func Cetak5Terbaru() {
    limit := 5
    if nPustaka < 5 {
        limit = nPustaka
    }
    var i int
    i = 0
    for i < limit {
        fmt.Println(pustaka[i].judul)
        i = i + 1
    }
}

func CariBuku(r int) {
    low := 0
    high := nPustaka - 1
    ketemu := -1
    for low <= high {
        mid := (low + high) / 2
        if pustaka[mid].rating == r {
            ketemu = mid

```

```

        break
    } else if pustaka[mid].rating > r {
        low = mid + 1
    } else {
        high = mid - 1
    }
}
if ketemu == -1 {
    fmt.Println("Tidak ada buku dengan rating seperti itu")
} else {
    b := pustaka[ketemu]
    fmt.Println(b.judul, b.penulis, b.penerbit, b.tahun,
b.eksemplar, b.rating)
}
}

func main() {
    DaftarkanBuku()
    CetakTerfavorit()
    UrutBuku()
    Cetak5Terbaru()
    var r int
    fmt.Scan(&r)
    CariBuku(r)
}

```

Output :

```

PS C:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-Razak\Week13-modul14\Unguided> go run "c:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-Razak\Week13-modul14\Unguided\insertion2.go"
3
B001 HarryPotter Rowling Gramedia 10 1997 95
B002 CleanCode Martin O'Reilly 5 2008 88
B003 Dune Herbert Chilton 7 1965 72
HarryPotter Rowling Gramedia 1997
HarryPotter
CleanCode
Dune
88
CleanCode Martin O'Reilly 2008 5 88
PS C:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-Razak\Week13-modul14\Unguided>

```

Penjelasan :

Kode diatas berperan seperti perpustakaan

3. Soal Unguided (selection) 1

Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu. Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program rumahkerabat yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort. Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat. Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masing-masing daerah.

```
package main

import "fmt"

type arrInt [1000000]int

func selectionSort(T *arrInt, n int) {
    var t, i, j, idx_min int
    i = 1
    for i <= n-1 {
        idx_min = i - 1
        j = i
        for j < n {
            if T[idx_min] > T[j] {
                idx_min = j
            }
            j = j + 1
        }
        t = T[idx_min]
        T[idx_min] = T[i-1]
        T[i-1] = t
        i = i + 1
    }
}

func main() {
    var n, m int
    fmt.Scan(&n)

    var i int
    i = 0
```

```

    for i < n {
        fmt.Scan(&m)
        var arr arrInt
        var j int
        j = 0
        for j < m {
            fmt.Scan(&arr[j])
            j = j + 1
        }
        selectionSort(&arr, m)
        var k int
        k = 0
        for k < m {
            if k > 0 {
                fmt.Print(" ")
            }
            fmt.Print(arr[k])
            k = k + 1
        }
        fmt.Println()
        i = i + 1
    }
}

```

Output :

```

PS C:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-Razak\Week13-modul14\Unguided> go run "c:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-Razak\Week13-modul14\Unguided\selection1.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1

```

Penjelasan :

Kode diatas bertujuan cari nilai terkecil, tukar ke posisi paling kiri secara berulang

4. Soal Unguided (selection) 2

Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program kerabat dekat yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut

membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

```
package main

import "fmt"

type arrInt [1000000]int

func selectionSortAsc(T *arrInt, n int) {
    var t, i, j, idx_min int
    i = 1
    for i <= n-1 {
        idx_min = i - 1
        j = i
        for j < n {
            if T[idx_min] > T[j] {
                idx_min = j
            }
            j = j + 1
        }
        t = T[idx_min]
        T[idx_min] = T[i-1]
        T[i-1] = t
        i = i + 1
    }
}

func selectionSortDesc(T *arrInt, n int) {
    var t, i, j, idx_max int
    i = 1
    for i <= n-1 {
        idx_max = i - 1
        j = i
        for j < n {
            if T[idx_max] < T[j] {
                idx_max = j
            }
            j = j + 1
        }
        t = T[idx_max]
        T[idx_max] = T[i-1]
        T[i-1] = t
        i = i + 1
    }
}
```



```

}

func main() {
    var n, m int
    fmt.Scan(&n)

    var i int
    i = 0
    for i < n {
        fmt.Scan(&m)
        var ganjil, genap arrInt
        var ng, nge int
        ng = 0
        nge = 0
        var j int
        j = 0
        for j < m {
            var x int
            fmt.Scan(&x)
            if x%2 != 0 {
                ganjil[ng] = x
                ng = ng + 1
            } else {
                genap[nge] = x
                nge = nge + 1
            }
            j = j + 1
        }

        selectionSortAsc(&ganjil, ng)
        selectionSortDesc(&genap, nge)

        first := true
        var k int
        k = 0
        for k < ng {
            if !first {
                fmt.Print(" ")
            }
            fmt.Print(ganjil[k])
            first = false
            k = k + 1
        }
        k = 0
        for k < nge {

```

```

        if !first {
            fmt.Print(" ")
        }
        fmt.Print(genap[k])
        first = false
        k = k + 1
    }
    fmt.Println()
    i = i + 1
}
}

```

Output :

```

PS C:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-Razak\Week13-modul14\Unguided> go run "c:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-Razak\Week13-modul14\Unguided\selection2.go"
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS C:\Users\ALIF\Documents\Algoritma\109082530009_Alif-Ar-Razak\Week13-modul14\Unguided>

```

Penjelasan :

Sama kek soal 1, tapi saat membaca input, angka **ganjil** dan **genap** dipisah ke dua array berbeda. Ganjil di-sort ascending, genap di-sort descending. Saat cetak, ganjil dulu baru genap dalam satu baris

D. Kesimpulan

Kesimpulan nya Adalah semua kode diatas menggunakan Bahasa golang,